

Sisteme de Operare. Laborator 6

Honorius Gâlmeanu
galmeanu@unitbv.ro

April 2, 2023

1 Identificatori de proces

Fiecare proces este identificat cu ajutorul unei valori întregi pozitive, referită ca Process ID (PID). Procesul cu PID = 1 este procesul `systemd`, primul proces rulat pe sistem la boot-are, și este permanent activ atâta vreme cât sistemul funcționează. El devine procesul părinte al oricăror procese orfane.

Identificatorii asociați unui proces sunt returnați de funcțiile:

```
#include <unistd.h>

pid_t getpid (void);      // PID-ul procesului apelant
pid_t getppid(void);     // PID-ul părintelui procesului apelant

uid_t getuid (void);     // real user ID-ul procesului apelant
uid_t geteuid(void);     // effective user ID-ul procesului apelant

gid_t getgid (void);     // real group ID-ul procesului apelant
gid_t getegid(void);     // effective group ID-ul procesului apelant
```

Aceste funcții nu returnează coduri de eroare.

2 Funcția fork()

Apelul de sistem `fork()` creează un nou proces dintr-un proces existent. După terminarea execuției apelului controlul se întoarce atât în procesul părinte cât și în procesul nou creat. Din acel moment vom avea două procese identice ca și cod, cu observația că valoarea returnată în procesul copil va fi 0, în vreme ce valoarea returnată în procesul părinte este PID-ul procesului copil nou creat. Astfel, se duplică procesul apelant:

```
#include <unistd.h>

pid_t fork(void);
```

Procesul copil este o copie a procesului părinte; el preia o copie a spațiului de date (BSS), heap-ului și stivei procesului părinte. Cele două procese nu vor partaja decât zona de text (program), și doar dacă aceasta este read-only. Implementările Unix curente (de exemplu LINUX) nu realizează o copie integrală a zonelor de date, stivă și heap ale procesului părinte ci se va folosi o tehnică numită copy-on-write (COW). Aceste regiuni sunt partajate de părinte și de fiu dar vor fi marcate ca read-only. Dacă unul din procese încearcă să modifice aceste regiuni, kernelul va face o copie doar a zonei respective (de obicei o pagină de memorie).

O caracteristică a funcției `fork` este că toți descriptorii deschiși în procesul părinte sunt duplicați în procesul fiu (ca și când ar fi executat un apel `dup` pentru toți descriptorii). Este important ca procesul părinte și procesul fiu să dețină aceleași deplasamente în cadrul fișierelor deschise. Dacă ambele procese scriu în același fișier, fără a fi sincronizate într-un anumit mod, rezultatul va fi o scriere întrețesută, depinzând de ordinea de planificare (nedeterministă). Totuși nu acesta este modul de scriere cerut de majoritatea aplicațiilor. Există două modalități de manevrare a descriptorilor după apelul funcției `fork`:

- părintele așteaptă ca fiul să își termine execuția; fișierele partajate vor avea deplasamentele actualizate corespunzător cu ceea ce a setat fiul;
- ambele procese se vor executa normal; după `fork`, părintele și fiul vor închide descriptorii de care nu mai au nevoie.

În afară de fișierele deschise, procesul fiu va mai moșteni următoarele:

- real UID, real GID, effective UID, effective GID;
- process group ID, set-UID și set-GID;
- terminalul de control;
- directorul de lucru curent;
- directorul rădăcină;
- masca de creare a fișierelor;
- masca semnalelor;
- variabilele de environment;
- limitele resurselor.

Diferențele dintre procesul părinte și procesul fiu vor fi:

- valoarea returnată de `fork`;
- PID-urile sunt diferite;
- valorile corespunzătoare procesului fiu pentru `tms_utime`, `tms_stime`, `tms_cutime` și `tms_ustime` vor fi 0;
- file locks setate de procesul părinte nu vor fi moștenite de procesul fiu;
- pending alarms vor fi resetate pentru procesul fiu;
- mulțimea pending signal pentru fiu va fi mulțimea vidă.

Execuția apelului `fork` va eșua dacă:

- sunt deja prea multe procese active;
- numărul total de procese pentru real user ID-ul respectiv depășește limita maximă.

Apelul `fork` se utilizează de obicei când:

- un proces dorește ca el și procesul fiu să poată executa secțiuni de cod diferite în același timp; este cazul serverelor de rețea. Părintele așteaptă o cerere de servire de la un client; când aceasta sosește, părintele va apela `fork` și va lăsa fiului deservirea cererii; apoi părintele va aștepta o nouă cerere;
- un proces dorește să execute un alt program; este cazul shell-urilor. În acest caz fiul execută un apel `exec` imediat după apelul `fork`.

3 Funcțiile `exit`

Un proces se poate termina:

1. normal:

- execuția funcției `return` din `main`. Este echivalentă cu execuția funcției `exit` din `main`;
- apelul funcției `exit`; funcția include apelul funcțiilor `exit handler` (înregistrate prin apelarea `atexit`) și închiderea tuturor stream-urilor standard;

- apelul funcției `_exit`; această funcție este apelată de `exit`, va închide toți descriptorii de fișier ce aparțin procesului și care nu au fost închiși, iar procesele copil sunt moștenite de `init` sau de procesul specific definit în acest scop. Procesului părinte i se trimite un semnal `SIGCHLD`;

2. anormal:

- apelul `abort`, pentru care se generează semnalul `SIGABRT`;
- recepționarea anumitor semnale - de exemplu `SIGKILL`.

Pentru toate aceste cazuri este necesar ca procesul care își termină execuția să informeze procesul părinte despre modul de terminare. În cazul funcțiilor `exit` și `_exit` acest lucru este realizat prin apelarea acestora cu `exit status` ca argument. În cazul terminării anormale kernelul generează `termination status` indicând motivul terminării anormale. În oricare din cazuri procesul părinte poate obține `termination status` apelând funcția `wait` sau `waitpid`. `Exit status` este convertit în `termination status` de către kernel când este apelată funcția `_exit`.

Dacă procesul părinte își termină execuția înaintea procesului fiu, procesul fiu devine orfan și procesul `init` devine părintele său. Vom spune că procesul respectiv a fost moștenit de procesul `init` (`systemd`). De câte ori se termină un proces, kernelul inspectează procesele active pentru a vedea dacă procesul care tocmai s-a terminat nu este părintele unuia dintre ele. Dacă găsește un fiu al acestui proces, kernelul va modifica în 1 PID-ul părintelui procesului fiu. Astfel se garantează că fiecare proces are un părinte activ.

Un proces devine proces zombie dacă s-a terminat dar procesul părinte nu mai așteaptă să-i furnizeze modul de terminare. Codificarea stării zombie pentru un proces în cadrul comenzii `ps` este "Z".

4 Funcțiile `wait` și `waitpid`

Când se termină un proces, normal sau anormal, părintele său este atenționat de către kernel prin trimiterea semnalului `SIGCHLD`. Terminarea unui proces este un eveniment asincron, adică poate apare oricând în timpul execuției procesului părinte. Semnalul `SIGCHLD` generat de kernel către părinte reprezintă o notificare asincronă. Părintele poate ignora semnalul sau poate apela o funcție la apariția semnalului (signal handler). Implicit va ignora acest semnal.

Apelul funcțiilor `wait` și `waitpid` conduce la una din situațiile:

- blocarea procesului, dacă toți fiii săi sunt în execuție;
- returnarea imediată indicând starea de terminare a fiului, dacă un fiu și-a terminat execuția;
- returnarea imediată a unui cod de eroare, dacă nu mai există nici un proces fiu.

Dacă procesul va apela `wait`, deoarece a recepționat semnalul `SIGCHLD`, funcția `wait` va returna imediat și nu va bloca procesul. Dacă vom apela `wait` neținând cont de apariția semnalului `SIGCHLD`, procesul se va bloca.

```
#include <sys/wait.h>
```

```
pid_t wait    (int *wstatus);
pid_t waitpid(pid_t pid, int *wstatus, int options);
```

Diferențele dintre cele două funcții sunt:

- `wait` blochează procesul din care este apelată până când procesul fiu se termină; `waitpid` are o opțiune care previne blocarea procesului;
- `wait` așteaptă ca oricare din fii să-și termine execuția, în vreme ce `waitpid` poate specifica așteptarea terminării execuției pentru un anumit fiu.

Pentru ambele funcții `status` este un pointer la un întreg. Dacă acest argument este un pointer nenul, `termination status` corespunzător procesului fiu terminat este stocat la locația specificată de acest pointer. Dacă nu ne interesează `termination status` putem folosi ca argument pointerul `NULL`. Funcțiile vor returna PID-ul procesului fiu care și-a terminat execuția sau -1 în caz de eroare.

Există câteva macro-uri definite în `sys/wait.h` pentru examinarea `termination status`-ului returnat în `status`. Trei dintre ele sunt mutual exclusive și în funcție de rezultatul returnat se vor apela alte macro-uri pentru a determina `termination status`:

- `WIFEXITED(status)` - este "true" dacă starea a fost returnată de un proces care s-a terminat normal. În acest caz se poate executa `WEXITSTATUS(status)` pentru a prelua cei mai puțini semnificativi 8 biți ai argumentului pe care fiul l-a transmis funcției `exit` sau `_exit`;
- `WIFSIGNALED(status)` - este "true" dacă starea a fost returnată de un proces care s-a terminat anormal. În acest caz se poate executa `WTERMSIG(status)` pentru a prelua numărul semnalului care a cauzat terminarea. Mai poate fi apelat `WCOREDUMP(status)` care va returna "true" dacă a fost generat un fișier core; `WIFSTOPPED(status)` - este "true" dacă starea a fost returnată de un fiu care este trecut în starea stop. În acest caz putem executa `WSTOPSIG(status)` pentru a prelua numărul semnalului care a determinat trecerea procesului în starea stop.

Argumentul `pid` al funcției `waitpid` poate lua următoarele valori:

- `pid == -1` va aștepta terminarea oricărui proces fiu; în acest caz funcția se comportă ca și `wait`;
- `pid > 0` așteaptă terminarea procesului cu `pid`-ul specificat;
- `pid == 0` așteaptă terminarea oricărui proces fiu al cărui proces group ID este egal cu cel al procesului părinte;
- `pid < -1` așteaptă terminarea oricărui proces fiu al cărui proces group ID este egal cu valoarea absolută a `pid`.

Valoarea argumentului `options` este 0 sau un "sau" logic între constantele:

- `WNOHANG`: `waitpid` nu se va bloca dacă fiul specificat prin `pid` nu este imediat disponibil. În acest caz funcția va returna 0;
- `WUNTRACED`: se va realiza return chiar și pentru fiii care au trecut în starea stop și a căror stare nu a fost raportată.

În concluzie `waitpid` asigură trei funcționalități suplimentare față de `wait`:

1. `waitpid` permite specificarea procesului fiu pentru care se așteaptă preluarea stării;
2. `waitpid` poate implementa o versiune neblokantă a funcției `wait`;
3. `waitpid` asigură suport pentru controlul joburilor prin opțiunea `WUNTRACED`.

5 Funcțiile exec

Când un proces apelează una din funcțiile `exec`, acel proces va fi complet înlocuit de noul program. Noul program își va începe execuția cu propria funcție `main`.

PID-ul nu se va modifica deoarece nu se creează un proces nou; `exec` realizează numai înlocuirea procesului curent i.e. zona sa de date, heap și stivă, cu un nou program. Cu ajutorul funcției `fork` putem crea procese noi, iar cu funcțiile `exec` putem iniția programe noi. Aceste funcții împreună cu funcțiile `exit` și cele două funcții `wait` sunt singurele primitive de care avem nevoie pentru controlul proceselor.

```
#include <unistd.h>
extern char **environ;

int execl (const char *pathname, const char *arg, ...
           /*, (char *) NULL */);
int execlp (const char *file, const char *arg, ...
           /*, (char *) NULL */);
int execle (const char *pathname, const char *arg, ...
           /*, (char *) NULL, char *const envp[] */);
int execlv (const char *pathname, char *const argv[]);
int execlvp (const char *file, char *const argv[]);
int execlvpe (const char *file, char *const argv[], char *const envp[]);
```

Literele de la sfârșitul numelui funcției sunt în concordanță cu argumentele funcției. Litera "p" înseamnă că funcția primește ca argument un nume de fișier și utilizează variabila de mediu PATH pentru a găsi fișierul executabil. Funcțiile care nu au litera "p" în nume vor primi ca argument numele fișierului incluzând calea către acesta. Litera "l" înseamnă că argumentele liniei de comandă pentru programul specificat se vor furniza sub formă de listă. În cazul literei "v" aceste argumente se vor specifica sub formă de vector i.e. `argv[]`. Litera "e" înseamnă că funcția va folosi un alt mediu decât cel curent și va fi specificat printr-un pointer la masivul de pointeri către environment string, adică `envp[]`.

Noul proces va moșteni de la procesul apelant următoarele:

- PID-ul său și PID-ul părintelui;
- real UID și real GID;
- process group ID;
- session ID;
- terminalul de control;
- timpul rămas până la alarm clock;
- directorul de lucru curent;
- directorul rădăcină;
- masca de creare a fișierelor;
- file locks;
- masca semnalelor;
- pending signals;
- limitările resurselor;
- `tms_utime`, `tms_stime`, `tms_cutime` și `tms_ustime`.

În majoritatea sistemelor numai una dintre aceste funcții, `execv`, este un apel de sistem. Celelalte funcții sunt doar funcții de bibliotecă. Ele vor apela funcția de sistem `execv`.

6 Exemple

Pentru a putea compila programele date ca exemplu va fi necesar ca mai întâi să se construiască biblioteca `liblab6.a`. Acesta se face cu comanda:

```
Command Line
```

```
make lib
```

După generarea bibliotecii `liblab6.a`, se compilează exemplele cu:

```
Command Line
```

```
make
```

6.1 Programul fork1

Programul exemplifică utilizarea funcției `fork`. El se execută astfel:

```
$ ./fork1
$ ./fork1 > temp.out
$ cat temp.out
```

Explicați diferențele constatate în ieșirea programului pentru cele două execuții (cu și fără redirectare).

6.2 Programul wait1

Programul exemplifică utilizarea funcției `wait` și a macro-urilor de preluare a termination status. Programul se va executa astfel:

```
$ ./wait1
```

6.3 Programul fork2

Se exemplifică modul prin care se evită crearea proceselor zombie, prin preluarea fiului orfan de către `init`. Programul se apelează astfel:

```
$ ./fork2
```

6.4 Programul tellwait1

Se exemplifică apariția concurenței între procese. Programul va afișa un șir de caractere de la procesul părinte și un șir de caractere de la procesul fiu. Programul se apelează astfel:

```
$ ./tellwait1
```

Rulați de mai multe ori, pentru că rezultatul întrețeserii afișării este nedeterminist.

6.5 Programul tellwait2

Se exemplifică evitarea concurenței între procese apărută în programul `tellwait1`. Se vor utiliza funcțiile `TELL_xxx` și `WAIT_xxx` ce se bazează pe comunicarea cu semnale. Programul se apelează astfel:

```
$ ./tellwait2
```

6.6 Programul exec1

Se exemplifică utilizarea funcțiilor `exec`. Programul va lansa în execuție un alt program denumit `echoall`. Programul se apelează astfel:

```
$ ./exec1
```

7 Exerciții

Exercițiul 1

Rulați toate programele prezentate. Asigurați-vă că le-ați înțeles funcționarea. Folosiți-vă de pagina 2 de manual (comanda `man`).

Exercițiul 2

Scrieți un program care să implementeze un shell minimal. Shell-ul va trebui să citească de la standard input comenzi și să le execute.

Exercițiul 3

Modificați programul `tellwait2` astfel încât procesul fiu să fie executat primul.